

LCEL 核心		LCEL Core
管道操作符		
chain = prompt llm parser	用 串 Runnable	
chain.invoke({...})	同步调用	
await chain.ainvoke(x)	异步调用	
chain.stream(x)	流式输出 token	
chain.batch([x1,x2])	批量并发	
Runnable 原语		
RunnableSequence	顺序串联(即此)	
RunnableParallel	并行多分支,dict 输出	
RunnablePassthrough	透传输入,配 .assign()	
RunnableLambda(fn)	把函数变 Runnable	
RunnableBranch	条件分支 if/else	
实用方法		
.with_retry()	自动重试	
.with_fallbacks([...])	兜底链(主链失败转备链)	
.with_config(tags=, run_name=)	追踪标签 / LangSmith 命名	
.with_types(input_type=, output_type=)	显式类型,Server 校验用	
.assign(field=Runnable)	向 dict 输出加新字段	
.pick("key") / .pick(["a","b"])	取 dict 子集	

Models		Chat Models & Embeddings
Chat Models		
ChatOpenAI	langchain-openai	
ChatAnthropic	langchain-anthropic	
ChatGoogleGenerativeAI	Gemini	
ChatOllama	本地模型	
通用工厂(推荐)		
init_chat_model("openai:gpt-4o")	字符串选 provider	
init_chat_model("anthropic:claude-...")	动态切换提供方	
常用参数		
temperature=0	确定性输出	
max_tokens=1024	输出上限	
timeout / max_retries	网络保护	
绑定 / 配置		
llm.bind(stop=["\n"])	预绑定参数	
llm.bind_tools([...])	附加工具(function call)	
Embeddings		
OpenAIEmbeddings	text-embedding-3-*	
HuggingFaceEmbeddings	本地 sentence-transformers	
.embed_query(text)	单条 → 向量	
.embed_documents([...])	批量,RAG 入库用	

Prompts		Prompt Templates
基本模板		
PromptTemplate.from_template("Q: {q}")	单字符串模板	
{var}	变量插值占位	
Chat 模板		
ChatPromptTemplate.from_messages([...])	多角色消息	
("system", "你是...")	系统提示元组	
("human", "{question}")	用户消息	
("ai", "...")	AI 历史回复	
MessagesPlaceholder("history")	塞入历史消息列表	
Few-Shot		
FewShotPromptTemplate	字符串场景	
FewShotChatMessagePromptTemplate	Chat 场景	
example_selector=...	动态挑示例(语义/长度)	
局部赋值 / 组合		
prompt.partial(name="x")	先填一部分变量	
prompt.format(**vars)	直接渲染为字符串	
prompt.input_variables	查看待填字段	
prompt another_prompt	模板拼接(Chat 模板可叠加)	

Output Parsers		Parsers
基础解析器		
StrOutputParser()	取 .content 为字符串	
JsonOutputParser()	解析为 dict,支持流式	
CommaSeparatedListOutputParser	逗号分隔列表	
XMLOutputParser	XML 结构	
Pydantic schema		
PydanticOutputParser(pydantic_object=Cls)	老式,需手动注入 format 指令	
parser.get_format_instructions()	生成 schema 提示	
推荐:结构化输出		
llm.with_structured_output(Schema)	直接返回实例,首选	
method="json_schema" include_raw=True	原生 JSON schema 模式 同时拿到原始 message	
流式		
JsonOutputParser 配 .stream()	可产 partial JSON,UI 友好	
兜底		
OutputFixingParser	用 LLM 修复格式错误	
RetryOutputParser	带原 prompt 重试解析	

Retrievers / Vector Stores		Retrieval
向量库		
Chroma	本地嵌入式,开发首选	
FAISS	本地内存,无依赖	
PGVector	Postgres 扩展	
Pinecone	托管云服务	
Qdrant / Weaviate / Milvus	主流自托管	
变 Retriever		
vs.as_retriever(search_kwargs={"k":5})	默认 similarity	
search_type="mmr"	最大边际相关,提多样性	
search_type="similarity_score_threshold"	阈值过滤	
高级 Retriever		
MultiQueryRetriever	LLM 改写多问法再合并	
ParentDocumentRetriever	小块召回 / 大块返回	
ContextualCompressionRetriever	召回后压缩/重排	
EnsembleRetriever	BM25 + 向量混合	
SelfQueryRetriever	LLM 解析查询 → metadata 过滤	
TimeWeightedVectorStoreRetriever	最近访问加权(记忆类)	
Rerank		
CohereRerank	配 ContextualCompression 用	
FlashrankRerank	本地轻量交叉编码器	

Loaders & Splitters		Document I/O
文档加载器		
WebBaseLoader(url)	抓网页 HTML	
PyPDFLoader / PDFLoader	PDF	
DirectoryLoader(path, glob=...)	批量加载目录	
UnstructuredLoader	通吃多格式(DOCX/PPT/HTML)	
NotionDB / Confluence / GitHub	SaaS 接入	
CSVLoader / JSONLoader	结构化数据	
文本切分		
RecursiveCharacterTextSplitter	推荐,逐级分隔符	
chunk_size=1000, overlap=200	常用参数	
MarkdownHeaderTextSplitter	按 # 标题保结构	
TokenTextSplitter	按 tiktoken token 数	
Language.PYTHON	代码语义切分	
Document 对象		
Document(page_content, metadata)	是一容器,metadata 决定后续过滤	
语义切分(实验)		
SemanticChunker(embeddings)	按句子相似度断点	
breakpoint_threshold_type	percentile / standard_deviation	

Tools		Tool Calling
定义工具		
@tool	装饰器,从函数签名 + docstring 推 schema	
@tool(args_schema=Cls)	显式 Pydantic schema	
Tool.from_function(fn, name, desc)	命令式构造	
StructuredTool	多参数工具	
绑定到模型		
llm_with_tools = llm.bind_tools([t1,t2])	注入工具定义	
resp.tool_calls	读取模型决定要调的工具	
tool_choice="any"/"name"	强制选工具	
LangGraph(新 Agent 写法)		
ToolNode([t1,t2])	图节点自动执行 tool_calls	
create_react_agent(llm, tools)	prebuilt ReAct agent	
Legacy(过渡)		
AgentExecutor	老 API,推荐迁 LangGraph	
内置工具集		
TavilySearchResults · WikipediaQueryRun · PythonREPLTool · ShellTool · RequestsGetTool		
错误处理		
handle_tool_error=True	异常回喂模型继续	
InjectedToolArg	字段不暴露给 LLM,运行时注入	

RAG 套路 + Pro Tips		RAG Recipe & Tips
最小 RAG chain(LCEL)		
<pre>retriever = vs.as_retriever(search_kwargs={"k": 4}) chain = (RunnableParallel({ "context": retriever, "question": RunnablePassthrough(), }) prompt llm StrOutputParser()) chain.invoke("What is LCEL?")</pre>		
▶ 结构化输出用 with_structured_output(Schema),别再拼 PydanticOutputParser		
▶ 调试开 LangSmith: LANGSMITH_TRACING=true + API key		
▶ 生产优先 async(ainvoke/abatch/astream),并发扛 10x		
▶ 新 Agent 一律走 LangGraph, AgentExecutor 是 legacy		
▶ RAG 召回弱 → MMR + MultiQuery + Cohere/BGE rerank		
▶ 流式 UI: astream_events(version="v2") 拿细粒度事件		
▶ 别装 langchain 大包,只装 core + 需要的 partner		